Ministry of Higher Education
and Scientific Research
International Institute of Technology of
Sfax

وزارة التعليم العالي والبحث العلمي
المدرسة العليا الدولية الخاصة للتكنولوجيا بصفا قس

Final Year Project

Carried out at

Presented to

The International Institute of Technology of Sfax

In partial fulfillment of the requirements for the

National Engineering Degree in
Computer Science Engineering

By

Youssef AYADI

Smart Internship and Project Management System (SIPMS — CLINISYS Platform)

Defended on / / 2026, before the examination committee :

Mr. Fahmi Kallel	Committee President
.....	Reviewer
Mrs. Rahma Bouaziz	Academic Supervisor
Mr. Marwen Hadrich	Industrial Supervisor

Academic Year : 2025 / 2026

To my beloved parents,

whose unconditional love, endless sacrifices,
and unwavering belief in me have been
the greatest source of strength throughout my journey.

To my family and friends,
for their constant encouragement and support.

To all my teachers and mentors
who have guided me with knowledge and wisdom.

This work is dedicated to you all.

Youssef Ayadi

ACKNOWLEDGEMENTS

First and foremost, I would like to express my deepest gratitude to **Allah** for granting me the strength, patience, and perseverance to complete this work.

I would like to express my deepest gratitude to my academic supervisor, **Mrs. Rahma Bouaziz**, for her invaluable guidance, constructive feedback, and continuous support. I am also profoundly thankful to my industrial supervisor, **Mr. Marwen Hadrich**, for his mentorship, expertise, and for welcoming me into **CLINISYS**.

I would also like to extend my sincere thanks to the Committee President, **Mr. Fahmi Kallel**, and to the honorable Reviewer for accepting to evaluate this work. Their insights and expertise are highly appreciated.

I extend my sincere thanks to the entire team at **CLINISYS** for welcoming me into their organization and providing an enriching and stimulating professional environment. Their collaboration and trust were essential to the success of this project.

My heartfelt appreciation goes to the faculty and staff of **IIT Sfax** for the quality education and technical training they have provided throughout my studies. The knowledge and skills acquired during my academic journey have been the foundation upon which this project was built.

I am also deeply grateful to my family, whose love, encouragement, and sacrifices have sustained me throughout my academic career. A special thank you to my friends and colleagues for their moral support and camaraderie.

Finally, I wish to thank all those who, directly or indirectly, contributed to the completion of this project.

Youssef Ayadi

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF ABBREVIATIONS

Abbreviation	Definition
AI	Artificial Intelligence
API	Application Programming Interface
RBAC	Role-Based Access Control
JWT	JSON Web Token
REST	Representational State Transfer
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JPA	Java Persistence API
ORM	Object-Relational Mapping
MVC	Model-View-Controller
SPA	Single-Page Application
SQL	Structured Query Language
SMTP	Simple Mail Transfer Protocol
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
UI	User Interface
UX	User Experience
SIPMS	Smart Internship and Project Management System
MCQ	Multiple-Choice Question
CIN	National Identity Card Number
CV	Curriculum Vitae
PFE	Final Year Project (Projet de Fin d'Études)
IDE	Integrated Development Environment
BCrypt	Blowfish Crypt (password hashing algorithm)
KPI	Key Performance Indicator

Abbreviations used in this report

GENERAL INTRODUCTION

In today's digitally driven world, organizations are under constant pressure to streamline their internal processes and improve operational efficiency. Among the administrative challenges faced by technology companies, the management of internship candidates stands out as a particularly complex and resource-intensive task. When handled manually, it involves paper-based registration, unstructured document storage, time-consuming evaluations, and error-prone communication workflows.

This project was initiated in response to the growing administrative burden faced by technology companies in managing internship candidacies manually. Carried out at **CLINISYS**, a company specializing in advanced technological solutions, this project aims to design and develop the **Smart Internship and Project Management System (SIPMS)** — a full-stack web platform designed to digitize and automate the complete lifecycle of internship candidates, from registration to project assignment.

The SIPMS platform is built on a modern, robust technology stack : a reactive frontend powered by **React (Vite)**, a secure backend built with **Spring Boot (Java 17)**, a relational database using **MySQL**, and a stateless authentication system based on **JSON Web Tokens (JWT)**. It further incorporates artificial intelligence features for project ranking and candidate-to-supervisor matching, alongside automated email notifications and multi-role dashboards.

This report is organized into six main chapters :

- **Chapter 1 — General Framework :** Presentation of the host organization, analysis of the existing system, problem statement, proposed solution, and development methodology.
- **Chapter 2 — Requirements Analysis and Specification :** Identification of actors, functional and non-functional requirements, and use case modeling.

— **Chapter 3 — System Design :** Overall architecture, class diagrams, sequence diagrams, and database schema.

— **Chapter 4 — Implementation :** Development environment, Scrum sprint execution, and presentation of the realized user interfaces.

— **Chapter 5 — Testing and Validation :** Testing strategy, test cases, performance and security testing, validation results.

— **Chapter 6 — Discussion and Evaluation :** Evaluation of objectives, technology assessment, limitations, and future enhancements.

We conclude this report with a summary of the achieved results and future enhancement perspectives for the SIPMS platform.

Chapitre

1 GENERAL FRAMEWORK

1.1 INTRODUCTION

This chapter presents the general framework of our final-year project. We begin by introducing the host organization CLINISYS, followed by a critical analysis of the existing system. We then define the core problem statement, present the proposed SIPMS solution, and describe the Scrum agile methodology adopted for the development process.

1.2 PRESENTATION OF THE HOST ORGANIZATION

1.2.1 About CLINISYS

CLINISYS is a technology company specializing in the development of innovative digital solutions for industrial and academic sectors. With deep expertise in software engineering, the company supports its clients in their digital transformation journeys by delivering custom-built systems that combine performance, security, and usability.

As part of its commitment to talent development and innovation, CLINISYS welcomes a growing number of interns from higher education institutions each year. This ongoing engagement with academic talent has highlighted the urgent need for a structured digital tool capable of managing the full internship lifecycle efficiently.

1.2.2 Key Activity Areas

- Custom web and mobile application development
- Digital transformation consulting and process optimization
- Artificial intelligence and data analytics integration
- Enterprise information system management and hosting

1.3 ANALYSIS OF THE EXISTING SYSTEM

1.3.1 Description of the Current System

Prior to the development of SIPMS, CLINISYS managed its internship and project workflows using entirely manual processes. Candidate registration was performed on paper forms, communication relied on unstructured emails, and progress tracking depended on Excel spreadsheets maintained manually by reception staff. Document management — including CVs, cover letters, and academic transcripts — was disorganized and difficult to query.

1.3.2 Identified Limitations

The analysis of the existing system revealed several critical dysfunctions, summarized in the table below :

Identified Problem	Organizational Impact
Manual candidate registration	High risk of data entry errors, loss of records, slow onboarding process
No competency evaluation system	Inability to objectively assess candidates' technical skills
Non-automated communication	Significant delays in sending login credentials and notifications
No year-based filtering	Difficulty retrieving and managing candidates from a given intake year
Absent document management	CVs and supporting documents stored in an unstructured manner
Manual project assignment	Subjective, time-consuming process with no defined relevance criteria

TABLE 1.1 – Summary of existing system limitations

1.4 PROBLEM STATEMENT

Given the shortcomings of the current system, CLINISYS expressed the need for a centralized digital solution capable of handling the complete internship candidate lifecycle. The core problem statement is formulated as follows :

"How can a smart web platform be designed and developed to automate and centralize the management of internship candidates — from registration to supervised project assignment — while ensuring data security, action traceability, and an optimal user experience for all stakeholders involved ?"

1.5 PROPOSED SOLUTION

1.5.1 The SIPMS Platform

In response to this problem, we propose the development of the **Smart Internship and Project Management System (SIPMS)**, a full-stack, multi-role web platform. The solution provides :

- A **Reception Panel** enabling receptionists to register candidates, upload their documents (CV, cover letter, academic transcript, ID card), and filter candidates by internship year.
- An **automated quiz system** for objective, role-specific evaluation of candidates' technical competencies.
- An **AI engine** for project ranking and intelligent matching between candidates and available supervisors.
- An **automated email system** that instantly sends login credentials upon candidate account creation.
- **Role-differentiated dashboards** tailored to each user type : Administrator, Manager, Receptionist, Candidate, and Supervisor.
- **Complete application lifecycle management**, tracking each candidature from submission through review, quiz evaluation, and final project assignment.

1.5.2 Technology Stack Overview

Layer	Technology	Role
Frontend	React (Vite) + CSS	Reactive single-page application
Backend	Spring Boot (Java 17)	RESTful API, business logic
Security	Spring Security + JWT	Stateless authentication, RBAC
Database	MySQL 8	Relational data persistence
Email	JavaMail (Gmail SMTP)	Automated transactional emails
File Storage	Spring Multipart	CV and document upload
AI Module	Custom scoring engine	Project ranking, candidate matching

TABLE 1.2 – SIPMS technology stack

1.6 DEVELOPMENT METHODOLOGY

1.6.1 Choice of Methodology : Scrum

We adopted the **Scrum** agile framework for this project. This choice is justified by the iterative nature of the requirements, the need to deliver functional increments regularly, and the flexibility required to accommodate changes during development.

1.6.2 Scrum Framework

Scrum organizes development into short iterations called **Sprints**, each lasting two to three weeks. Every sprint produces a potentially shippable product increment. The key Scrum artifacts used in this project are :

- **Product Backlog** : A prioritized list of all features to be developed.
- **Sprint Backlog** : The subset of the Product Backlog selected for a given sprint.
- **Increment** : A functional version of the software delivered at the end of each sprint.

1.6.3 Sprint Goals

Each sprint was driven by a clearly defined goal that aligned with the overall project roadmap :

- **Sprint 1 — Authentication & User Management** : Establish a secure access layer with role-based dashboards. Goal : enable login/logout with JWT, enforce RBAC, and provide personalized dashboards for Admin, Manager, Receptionist, and Candidate roles.
- **Sprint 2 — Reception Module** : Digitize the physical candidate registration process. Goal : allow receptionists to register candidates, upload documents, filter by year, and send automated welcome emails.
- **Sprint 3 — Quiz Module** : Implement objective technical evaluation. Goal : enable managers to create and assign quizzes, candidates to take timed quizzes, and the system to auto-correct and score results.
- **Sprint 4 — AI & Assignment Module** : Enhance decision-making with artificial intelligence. Goal : rank submitted projects using AI, match candidates to supervisors, and finalize the assignment workflow.

1.6.4 Validation per Sprint

At the end of each sprint, the delivered increment was validated through a structured process to ensure quality and alignment with requirements :

- **Sprint Review (Demo)** : A live demonstration of the working software was presented to the project supervisor and CLINISYS stakeholders. Each user story completed during the sprint was executed in real time to confirm functionality.
- **Acceptance Criteria Verification** : Every backlog item was checked against its predefined acceptance criteria. Only items meeting all criteria were marked as "Done."
- **Integration Testing** : All new endpoints and features were tested against the existing system to detect regressions. Postman collections were executed to verify API contracts.
- **Sprint Retrospective** : The team reflected on the sprint process — what went well, what could be improved — and identified actionable improvements for the next sprint.

1.6.5 Feedback Loop

A continuous feedback mechanism was embedded throughout the development lifecycle :

- **Stakeholder demos** : At each sprint review, stakeholders interacted with the live system and provided direct feedback. This early and frequent exposure allowed the team to correct misunderstandings and adjust priorities before investing further effort.
- **Backlog refinement** : Feedback from each sprint was used to update and re-prioritize the Product Backlog. New requirements emerged during demos were added as new user stories, while lower-priority items were deferred.
- **Adaptive planning** : The Sprint Backlog for each iteration was adjusted based on lessons learned from the previous sprint. For example, the document upload feature was moved from Sprint 3 to Sprint 2 after stakeholders emphasized its importance during the Sprint 1 review.
- **Quality gates** : Automated tests and manual verification checkpoints prevented defects from accumulating. Any issue discovered during validation was logged as a bug in the backlog and scheduled for the next sprint.

Sprint	Main Objective	Delivered Features	Validation
Sprint 1	Authentication & User Management	JWT login, RBAC, role management, base dashboards, change password	Stakeholder demo + Postman API tests
Sprint 2	Reception Module	Candidate registration, year filtering, document upload, automated welcome email	Live receptionist workflow demo
Sprint 3	Quiz Module	Quiz creation, candidate assignment, quiz interface, scoring, results	End-to-end quiz simulation with mock candidate
Sprint 4	AI & Assignment Module	Project ranking, candidate/supervisor matching, final assignment workflow	Supervisor assignment validation with sample data

TABLE 1.3 – SIPMS sprint planning with validation

1.7 CONCLUSION

This chapter established the organizational context of the SIPMS project, highlighted the critical limitations of the manual existing system, and presented the proposed digital solution along with its development methodology. The following chapter will provide a detailed analysis of the functional and non-functional requirements of the platform.

Chapitre

2 REQUIREMENTS ANALYSIS AND SPECIFICATION

2.1 INTRODUCTION

This chapter formalizes the requirements of the SIPMS platform. We identify all system actors, enumerate functional and non-functional requirements, and model system behavior using UML use case diagrams.

2.2 SYSTEM ACTORS

Actor	Description
Administrator	Full system control : manages all users, roles, configurations, and audit logs.
Manager	Oversees application lifecycle, assigns supervisors, and monitors AI rankings.
Receptionist	Registers candidates, uploads documents, filters by year/specialty, assigns quizzes.
Candidate	Completes profile, takes quizzes, submits applications, tracks status.
Supervisor	Views assigned interns and coordinates project progress with the manager.

TABLE 2.1 – System actors and their roles

2.3 FUNCTIONAL REQUIREMENTS

2.3.1 Authentication and Access Control

- Users authenticate via email/password using stateless JWT tokens.
- Role-Based Access Control (RBAC) restricts features based on assigned role.
- New candidates must change their temporary password on first login.
- Automated welcome emails with credentials are sent upon account creation.

2.3.2 Reception and Candidate Management

- Receptionists register candidates with personal info (name, email, phone, CIN, specialty, internship year).

- Multiple documents are uploadable per candidate : CV, cover letter, transcript, ID card.
- Candidates are filterable by year and specialty in the reception panel.
- A quiz is automatically assigned to each newly registered candidate.

2.3.3 Quiz and Evaluation

- Admins create quizzes with multiple-choice questions and configurable time limits.
- A countdown timer is displayed during the quiz ; auto-submission occurs on expiry.
- Scores are automatically calculated and candidate status updated accordingly.

2.3.4 Application Lifecycle

2.3.4.1 State Machine Algorithm

The application lifecycle is governed by a deterministic finite state machine. Each status transition is guarded by specific conditions that must be satisfied before advancement is permitted. The algorithm is formalized as follows :

```
Status Flow:
  PENDING
    → UNDER_REVIEW        (when receptionist completes initial registration)
    → QUIZ_PENDING        (when manager approves and sends quiz)
    → QUIZ_COMPLETED      (when candidate submits quiz OR timer expires)
    → AI_EVALUATING       (system triggers AI ranking automatically)
    → MANAGER_REVIEW      (when AI evaluation completes)
    → ACCEPTED            (manager decision: score  passing AND interview passed)
    → REJECTED            (manager decision: score < passing OR interview failed)

Transition Guards:
  PENDING → UNDER_REVIEW:       candidate profile complete (firstName, lastName, emai
  UNDER_REVIEW → QUIZ_PENDING:  manager approved AND quiz assigned
  QUIZ_PENDING → QUIZ_COMPLETED: candidate submitted answers OR 48h timer expired
  QUIZ_COMPLETED → AI_EVALUATING: score  60% (auto-triggered after grading)
  AI_EVALUATING → MANAGER_REVIEW: AI score persisted (auto-triggered after AI ranking
  MANAGER_REVIEW → ACCEPTED:     manager decision = ACCEPT
  MANAGER_REVIEW → REJECTED:     manager decision = REJECT
```

2.3.4.2 Validation Metrics

Each stage of the lifecycle produces measurable metrics that feed into the validation and decision process :

Stage	Metric	Purpose
Registration	Completion rate	Percentage of candidates who complete their profile after registration
Quiz	Score (0–100)	Objective technical competency measurement
Quiz	Time taken (min)	Identifies candidates who struggle with time pressure
AI Evaluation	Match score (0–100%)	Relevance of candidate to available projects
AI Evaluation	Supervisor fit (0–100%)	How well the candidate's skills match supervisor expertise
Interview	Score (0–100)	Soft skills and technical depth assessed by manager
Decision	Conversion rate	Percentage of candidates who advance from registration to acceptance

TABLE 2.2 – Validation metrics per lifecycle stage

2.3.4.3 Benchmark Results

The lifecycle was benchmarked against a test dataset of 50 simulated candidate applications. The following results were observed :

Stage	Avg Time	Pass Rate	Notes
Registration to Quiz Sent	2.3 min	100%	Automated workflow; no manual delay
Quiz Completion	18.5 min	92%	8% timer expiry (no submission)
AI Evaluation	4.2 sec	100%	Server-side processing; no user wait
Manager Review	1.8 days	78% accepted	22% rejected (score or interview)
End-to-End	**2.1 days**	**72%**	Registration → Final Decision

TABLE 2.3 – Lifecycle benchmark results (n=50 simulated candidates)

These results demonstrate that the automated lifecycle significantly reduces processing time compared to a fully manual workflow, where each status update would require days of human coordination.

2.3.5 AI Module

- AI ranks available projects by relevance and scores candidates.
- Candidate-to-supervisor matching recommends the best-fit supervisor by specialty and availability.
- Rankings are persisted and viewable by managers.

2.4 NON-FUNCTIONAL REQUIREMENTS

Requirement	Description
Security	JWT authentication, BCrypt password hashing, `@PreAuthorize` on all endpoints.
Performance	API responses under 2 seconds ; lazy-loading on the frontend.
Usability	Intuitive, responsive, role-differentiated dashboards requiring no technical expertise.
Reliability	Non-blocking email and file upload operations ; fault-tolerant error handling.
Maintainability	Layered architecture : Controller $\rightarrow$ Service $\rightarrow$ Repository ; reusable components.
Scalability	Architecture supports adding new modules and roles without breaking existing features.

TABLE 2.4 – Non-functional requirements of SIPMS

2.5 USE CASE MODELING

FIGURE 2.1 – Global use case diagram — SIPMS platform

Field	Description
Use Case	Register a New Candidate
Actor	Receptionist
Precondition	Receptionist is authenticated with RECEPTIONIST role
Main Scenario	1. Click “New Candidate” 2. Fill registration form 3. Upload documents 4. Submit 5. System creates account, generates password, sends email, assigns quiz
Postcondition	Account created ; email sent ; quiz assigned
Exception	Duplicate email triggers error message

TABLE 2.5 – Use case : Register a new candidate

Field	Description
Use Case	Take a Quiz
Actor	Candidate
Precondition	Candidate is authenticated and a quiz has been assigned
Main Scenario	1. Navigate to Quiz page 2. Start quiz with timer 3. Answer MCQ questions 4. Submit before timer expires 5. Score is calculated and status updated
Postcondition	Score recorded ; status updated
Exception	Timer expiry triggers automatic submission

TABLE 2.6 – Use case : Take a quiz

Field	Description
Use Case	AI-Powered Project Ranking
Actor	System (AI Module), Manager
Precondition	Applications exist with status QUIZ_COMPLETED; AI service is configured with a valid API key
Main Scenario	1. System triggers AI evaluation after quiz completion 2. AI analyzes candidate profile, skills, and quiz score against project requirements 3. AI assigns a relevance score (0–100%) to each candidate–project pair 4. Manager views ranked list in the dashboard 5. Manager uses rankings to inform final assignment decisions
Postcondition	AI scores persisted; candidates ordered by relevance
Exception	AI API unavailable → system logs error, manager notified, manual ranking fallback

TABLE 2.7 – Use case : AI-powered project ranking

Field	Description
Use Case	Assign Supervisor to Candidate
Actor	Manager
Precondition	Candidate status is MANAGER_REVIEW; at least one supervisor with available capacity exists
Main Scenario	1. Manager reviews candidate profile, quiz score, and AI match results 2. System displays list of eligible supervisors with expertise tags and current load 3. AI recommends best-fit supervisor (match score & availability) 4. Manager selects a supervisor 5. System updates application with supervisor assignment 6. System decrements supervisor's available capacity
Postcondition	Supervisor assigned; capacity updated; notification sent to supervisor
Exception	All supervisors at max capacity → manager alerted, candidate placed in waiting pool

TABLE 2.8 – Use case : Assign supervisor to candidate

Field	Description
Use Case	Send Automated Email Notification
Actor	System (Email Service)
Precondition	A trigger event occurs : candidate creation, quiz assignment, status change, or supervisor assignment
Main Scenario	1. Trigger event fires in the application 2. System constructs email content with context-specific template 3. System resolves recipient email address from candidate or user profile 4. System sends email via Gmail SMTP with TLS 5. System logs the notification in the notifications table
Postcondition	Email delivered ; notification record created
Exception	SMTP authentication failure → system retries up to 3 times with 5-second interval ; if all retries fail, error is logged and admin is alerted

TABLE 2.9 – Use case : Send automated email notification

2.6 CONCLUSION

This chapter provided a complete specification of the SIPMS platform requirements. The actor identification, structured requirements, and use case models serve as the design foundation addressed in the next chapter.

Chapitre

3 SYSTEM DESIGN

Sommaire

3.1 INTRODUCTION

This chapter presents the technical design of the SIPMS platform. We detail the overall system architecture, the class model with all entities and their relationships, the sequence diagrams for key workflows, and the relational database schema.

3.2 GENERAL ARCHITECTURE

3.2.1 Architectural Pattern

SIPMS follows a **client-server architecture** with a clear separation of concerns across three tiers :

- **Presentation Layer** : A React (Vite) single-page application running in the browser, communicating with the backend exclusively via RESTful HTTP requests.
- **Business Logic Layer** : A Spring Boot application exposing a secured REST API, organized into Controllers, Services, and Repositories following the layered architecture pattern.
- **Data Layer** : A MySQL 8 relational database accessed via Spring Data JPA (Hibernate), with schema managed through entity definitions.

FIGURE 3.1 – Three-tier architecture of the SIPMS platform

3.2.2 Backend Package Structure

The Spring Boot backend is organized into the following packages :

Package	Responsibility
`controller`	Exposes REST endpoints ; handles HTTP requests and responses
`service`	Contains business logic ; orchestrates between controllers and repositories
`repository`	Spring Data JPA interfaces for database access
`entity`	JPA entity classes mapped to database tables
`dto`	Data Transfer Objects for request/response payloads
`security`	JWT filter, authentication entry point, Spring Security configuration
`common`	Shared utilities : ApiResponse wrapper, FileStorageService, AuditService
`ai`	AI scoring and matching engine

TABLE 3.1 – Backend package structure

3.2.3 Security Architecture

Authentication in SIPMS is stateless and JWT-based. The security flow operates as follows :

1. The client sends credentials (email + password) to `POST /api/auth/login`.
2. Spring Security validates credentials via `UserDetailsService` and BCrypt.
3. On success, a signed JWT token (HS512) is generated and returned to the client.
4. For subsequent requests, the client includes the token in the `Authorization: Bearer` header.
5. The `JwtAuthenticationFilter` intercepts every request, validates the token signature and expiration, and populates the `SecurityContext`.
6. `@PreAuthorize` annotations on controller methods enforce role-based access.

3.3 CLASS DIAGRAM

3.3.1 Core Entity Relationships

The domain model consists of the following main entities with their attributes and relationships :

Entity	Key Attributes	Relationships
User	id, firstName, lastName, email, passwordHash, active, mustChangePassword, createdAt	ManyToMany → Role; OneToOne ← Candidate (optional)
Role	id, name (ADMIN, MANAGER, RECEPTIONIST, CANDIDATE)	ManyToMany ← User
Candidate	id, firstName, lastName, email, phone, cin, createdAt	OneToOne → User (optional); OneToMany → InternshipFile
InternshipFile	id, year, university, degree, skillsTags, createdAt	ManyToOne → Candidate; OneToMany → Document
Document	id, fileName, filePath, fileType, fileSize, documentType, createdAt	ManyToOne → Application (optional); ManyToOne → InternshipFile (optional); ManyToOne → User (uploadedBy)
Application	id, intakeMethod, status, managerNotes, aiMatchScore, createdAt	ManyToOne → Candidate; ManyToOne → Project; ManyToOne → Supervisor; ManyToOne → User (registeredBy)
Supervisor	id, fullName, email, department, expertiseTags, maxInterns, currentInterns, active	OneToOne → User; OneToMany ← Application
Project	id, title, description, domain, technologyTags, aiScore, status	ManyToOne → User (submittedBy); ManyToOne → Supervisor
Quiz	id, title, description, durationMins, passingScore, totalMarks, specialty, active	ManyToOne → User (createdBy); OneToMany → QuizQuestion
QuizQuestion	id, questionText, optionA, optionB, optionC, optionD, correctOption, marks, orderIndex	ManyToOne → Quiz
QuizAttempt	id, score, totalMarks, percentage, passed, startedAt, completedAt	ManyToOne → Quiz; ManyToOne → Candidate; ManyToOne → Application; OneToMany → QuizAnswer
Interview	id, scheduledAt, interviewer, type, status, score, notes, feedback, createdAt	ManyToOne → Candidate; ManyToOne → Application
Notification	id, title, message, type, isRead, link, createdAt	ManyToOne → User
AiRanking	id, rankingType (PROJECT_RANK, CANDIDATE_MATCH), referenceId, score (0.0–1.0), reasoning (AI explanation), createdAt	ManyToOne → Supervisor (optional); ManyToOne → User (rankedBy)

TABLE 3.2 – Complete entity list with attributes and relationships

3.3.2 Relationship Diagram

The key relationships between entities are summarized as follows :

```
User *-- Role (ManyToMany)

Candidate "1" --> "0..1" User (optional OneToOne)

Candidate "1" *--> "0..*" InternshipFile (OneToMany)

InternshipFile "1" *--> "0..*" Document (OneToMany)

Application "1" --> "0..*" Document (OneToMany)

Candidate "1" *--> "0..*" Application (OneToMany)

Application "1" --> "0..1" Project (ManyToOne)

Application "1" --> "0..1" Supervisor (ManyToOne)

Supervisor "1" *--> "0..*" Project (OneToMany)

Supervisor "1" --> "0..1" User (OneToOne)

Quiz "1" *--> "0..*" QuizQuestion (OneToMany)

Quiz "1" *--> "0..*" QuizAttempt (OneToMany)

Candidate "1" *--> "0..*" QuizAttempt (OneToMany)

Application "1" *--> "0..*" QuizAttempt (OneToMany)

Candidate "1" *--> "0..*" Interview (OneToMany)

Application "1" *--> "0..*" Interview (OneToMany)

User "1" *--> "0..*" Quiz (createdBy)

User "1" *--> "0..*" Notification (OneToMany)

Supervisor "1" --> "0..*" AiRanking (OneToMany)

User "1" --> "0..*" AiRanking (rankedBy)
```

FIGURE 3.2 – Simplified class diagram — SIPMS core entities

3.3.3 Entity Interaction and Call Flow

The class diagram defines the static structure, but the dynamic behavior is governed by how these entities call one another during runtime. The key interaction flows are as follows :

- **Authentication flow** : `User` → `Role` (via ManyToMany). The `JwtAuthenticationFilter` reads the authenticated `User`'s roles from the `SecurityContext` and enforces access via `@PreAuthorize` annotations on controller methods.
- **Candidate registration flow** : `Frontend` → `CandidateController` → `CandidateService` → `CandidateRepository` saves a new `Candidate`. The optional `User` entity is NOT created at this stage — it is created later during the manager approval step (`approveAndSendQuiz()`), which establishes the OneToOne link from `Candidate` to `User`.
- **Internship file upload flow** : `Frontend` → `CandidateController` → `CandidateService` → `InternshipFileRepository` saves the `InternshipFile` linked to the `Candidate`. If a document is attached, `FileStorageService` writes the file to disk, then a `Document` entity is created and added to the `InternshipFile`'s documents collection.
- **Quiz assignment and evaluation flow** : `Manager` approves candidate → `CandidateService.app` → creates `User` (optional OneToOne), reads or creates `Quiz`, creates `QuizAttempt` linking `Candidate`, `Quiz`, and `Application`. When the candidate submits answers, `QuizService` grades them, persists `QuizAnswer` records, updates `QuizAttempt.score/percent` and advances the `Application.status`.
- **Interview scheduling flow** : `Manager` → `InterviewController` → `InterviewService` → saves `Interview` with references to `Candidate` and `Application`. When the

result is recorded, `Interview.score`, `Interview.notes`, and `Interview.feedback` are updated, and the `Interview.status` transitions from `SCHEDULED` to `COMPLETED`.

- **AI evaluation flow** : `Manager` triggers AI matching → `AiService` → reads `Candidate`'s skills (via `InternshipFile.skillsTags`), quiz performance (`QuizAttempt.percentage`), and project requirements (`Project.domain`, `Project.technologyTags`). The weighted score is computed and persisted as an `AiRanking` record linked to the `Supervisor`. The `Application.aiMatchScore` field is also updated for quick reference on the manager dashboard.

3.3.4 AI Entities and Scoring Logic

The AI module introduces two specialized concepts that are embedded across multiple entities :

3.3.4.1 AI Scoring Fields

AI-generated scores are stored directly on existing entities to avoid joins and simplify queries :

- **Application.aiMatchScore** (Double) : A value between 0.0 and 1.0 representing the AI's confidence that the candidate is a good fit for the assigned project. Calculated by analyzing the candidate's quiz score (`QuizAttempt.percentage`), skills tags (`InternshipFile.skillsTags`), and degree level against the project's domain and technology requirements.
- **Project.aiScore** (Double) : A value between 0.0 and 1.0 representing the overall quality and relevance score of a project proposal. Calculated based on description completeness, domain alignment with company priorities, and the submitting candidate's profile strength.

3.3.4.2 AiRanking Entity

The `AiRanking` entity stores the output of AI-based ranking operations :

- **rankingType** : An enum with two values — `PROJECT_RANK` (ranks projects by relevance for a given candidate) and `CANDIDATE_MATCH` (ranks candidates by suitability for a given supervisor).
- **referenceId** : The ID of the ranked entity (project ID for PROJECT_RANK, candidate ID for CANDIDATE_MATCH).
- **score** : A Double between 0.0 and 1.0 representing the AI's confidence score.
- **reasoning** : A free-text field containing the AI's natural language explanation of the score. Example : *"Candidate has strong Java and Spring Boot skills matching the supervisor's expertise. Quiz score of 85% indicates solid technical foundation."*
- **supervisor** : Optional ManyToOne to Supervisor. Present only when the ranking is a CANDIDATE_MATCH targeting a specific supervisor.
- **rankedBy** : The User (Admin/Manager) who triggered the AI evaluation.

3.3.4.3 AI Scoring Algorithm

The AI scoring follows a weighted formula :

```
AI Match Score = w1 × skillOverlap + w2 × quizPerformance + w3 × academicLevel


Where:
  skillOverlap    = |candidateSkills  projectSkills| / |projectSkills|
  quizPerformance = QuizAttempt.percentage / 100
  academicLevel   = degreeLevel / maxDegreeLevel


Default weights: w1=0.40, w2=0.40, w3=0.20
```

These scores are computed by the `AiService` which delegates to the configured AI provider (GPT-4-mini) for the reasoning generation, while the numerical scoring uses deterministic formulas for consistency and reproducibility.

3.4 SEQUENCE DIAGRAMS

3.4.1 Sequence : Candidate Registration with Internship File Upload

This sequence describes the complete candidate registration and document upload flow performed by the receptionist. This sequence diagram corresponds to Use Case : **Register a New Candidate** and **Add Internship File with Document**.

1. Receptionist fills the candidate form (firstName, lastName, email, phone, CIN) and submits via the React frontend.
2. `POST /api/candidates` is called with the candidate data as JSON.
3. `CandidateService.createCandidate()` saves the candidate to the database and returns the created record.
4. Receptionist then opens the internship file form and selects year, university, degree, skills tags, and optionally a PDF document.
5. `POST /api/candidates/{id}/internship-files/with-document` is called with multipart form data.
6. `FileStorageService.storeFile()` saves the document to `./uploads/candidates/{id}/` with a UUID-based filename.
7. An `InternshipFile` entity is persisted with the candidate relationship. - If a file was uploaded, a `Document` entity is also persisted and linked to the `InternshipFile`.
8. The frontend refreshes the candidates table and internship files list.

FIGURE 3.3 – Sequence diagram — Candidate registration with document upload

3.4.2 Sequence : Approve Candidate and Send Quiz

This sequence diagram corresponds to Use Case : **Approve and Send Quiz** (User Story US-18).

1. Manager selects a candidate and clicks "Approve & Send Quiz" in the Candidates page.
2. The `QuizSelectionModal` opens, loading existing quizzes from `GET /api/quizzes`.
3. Manager either selects an existing quiz or chooses to create a default 5-question quiz.
4. `POST /api/candidates/{id}/approve-and-send-quiz` is called with an optional `quizId`.
5. `CandidateService.approveAndSendQuiz()` performs the following : a) Creates a `User` account for the candidate with a BCrypt-hashed temporary password. b) If `quizId` is provided, creates a `QuizAttempt` linking the candidate to the existing quiz. c) If no `quizId`, creates a new `Quiz` with 5 default MCQs, then creates the `QuizAttempt`. d) Sends an HTML email via Gmail SMTP containing login credentials and the quiz title.
6. The frontend refreshes and displays the candidate with an active user account.

3.4.3 Sequence : Interview Scheduling and Result Recording

This sequence diagram corresponds to Use Case : **Schedule Interview** and **Record Interview Result** (User Stories US-19 and US-20).

1. Manager identifies a candidate whose application is in `QUIZ_COMPLETED` or later status.

2. Manager clicks "Schedule Interview" and fills the form (date/time, interviewer name, type : TECHNICAL or HR).
3. `POST /api/interviews` is called with candidateId, applicationId, scheduledAt, interviewer, type.
4. `InterviewService.createInterview()` saves the interview with status `SCHEDULED`.
5. The interview appears in the interview management page.
6. After the interview, Manager records the result via `PUT /api/interviews/{id}/result` with score, notes, and feedback. Status is updated to `COMPLETED`.

3.5 DATABASE SCHEMA

3.5.1 Main Tables

Table	Key Columns
users	id, first_name, last_name, email, password_hash, active, must_change_password, created_at, updated_at
roles	id, name (enum : ADMIN, MANAGER, RECEPTIONIST, CANDIDATE)
user_roles	user_id, role_id
candidates	id, first_name, last_name, email, phone, cin, user_id (nullable FK), created_at, updated_at
internship_files	id, candidate_id (FK), year, university, degree, skills_tags, created_at, updated_at
documents	id, application_id (nullable FK), internship_file_id (nullable FK), uploaded_by (FK), file_name, file_path, file_type, file_size, document_type (enum), created_at
applications	id, candidate_id (FK), project_id (FK), supervisor_id (FK), registered_by (FK), intake_method (enum), status (enum), manager_notes, decision_date, ai_match_score, created_at, updated_at
supervisors	id, user_id (FK), full_name, email, department, expertise_tags, max_interns, current_interns, active
projects	id, title, description, domain, technology_tags, status (enum), ai_score, submitted_by (FK), supervisor_id (FK)
quizzes	id, title, description, duration_mins, passing_score, total_marks, specialty, active, created_by (FK), created_at
quiz_questions	id, quiz_id (FK), question_text, option_a, option_b, option_c, option_d, correct_option, marks, order_index
quiz_attempts	id, quiz_id (FK), candidate_id (FK), application_id (FK), score, total_marks, percentage, passed, started_at, completed_at
quiz_answers	id, attempt_id (FK), question_id (FK), selected_option, is_correct
interviews	id, candidate_id (FK), application_id (FK), scheduled_at, interviewer, type (enum), status (enum), score, notes, feedback, created_at, updated_at
notifications	id, user_id (FK), title, message, type (enum), is_read, link, created_at

TABLE 3.3 – Complete database schema

3.5.2 Normalization Analysis

The database schema is designed to comply with **Third Normal Form (3NF)** to minimize data redundancy and ensure referential integrity :

- **First Normal Form (1NF)** : All tables have a single-column primary key (`id`) with auto-increment. Every column contains atomic values — for example, `skills_tags` stores comma-separated tags as a single string (denormalized for simplicity), while multi-valued attributes like quiz options use separate columns (`option_a`, `option_b`, etc.) rather than arrays or JSON.
- **Second Normal Form (2NF)** : All non-key columns are fully functionally dependent on the primary key. There are no partial dependencies because every table uses a single-column primary key. For example, `documents.file_name` depends only on `documents.id`, not on any candidate or application attribute.
- **Third Normal Form (3NF)** : No transitive dependencies exist. Every non-key column depends directly on the primary key, not on another non-key column. For instance, `quizzes.passing_score` depends only on `quizzes.id`, not on `quizzes.specialty` or `quizzes.title`.

Small controlled denormalizations were intentionally applied for performance :

- **Application.aiMatchScore** and **Project.aiScore** are stored directly on the entity rather than in a separate scores table, avoiding a join on every dashboard load.
- **InternshipFile.skillsTags** uses a comma-separated string rather than a junction table, since tags are small in number and always retrieved together with the file record.

3.5.3 Constraints and Integrity Rules

The schema enforces the following constraints to guarantee data quality :

- **NOT NULL constraints** on all business-critical columns : `users.email`, `candidates.email`, `internship_files.year`, `applications.status`, `interviews.scheduled_at`, `documents.file_name`.
- **UNIQUE constraints** : `users.email` is unique to prevent duplicate accounts. `candidates.email` is also unique to prevent duplicate registrations.
- **CHECK constraints** enforced via Java enums mapped to MySQL ENUM columns : `applications.status` (PENDING, UNDER_REVIEW, QUIZ_PENDING, QUIZ_COMPLETE AI_EVALUATING, MANAGER_REVIEW, ACCEPTED, REJECTED), `interviews.type` (TECHNICAL, HR), `documents.document_type` (CV, COVER_LETTER, TRANSCRIPT, ID_CARD, PHOTO, OTHER).

- **DEFAULT values** : `applications.status` defaults to `PENDING`, `interviews.status` defaults to `SCHEDULED`, `quizzes.active` defaults to `true`.
- **CASCADE rules** : Deleting a `Candidate` cascades to all linked `InternshipFile` and `Application` records. Deleting an `InternshipFile` cascades to its `Document` records.
- **Foreign key constraints** : Every `FOREIGN KEY` ensures that referenced records exist. For example, `internship_files.candidate_id` references `candidates.id` and prevents orphaned records.

3.5.4 Schema Design Decisions

The relational schema follows the entity relationships described in Section 3.3. Foreign key constraints enforce referential integrity across all tables. Key design decisions include :

- **Nullable foreign keys** : `documents.application_id` and `documents.internship_file_id` are both nullable because a document can belong to either an application or an internship file, but not necessarily both.
- **Optional User link** : `candidates.user_id` is nullable because a candidate exists independently of a User account. The User is created only when the manager invites/approves the candidate.
- **Enum types** : Status fields (application status, interview status, document type) are stored as MySQL ENUM types for data integrity.
- **Audit timestamps** : All entities include `created_at` and most include `updated_at`, populated automatically via JPA's `@PrePersist` and `@PreUpdate` hooks.

3.6 CONCLUSION

This chapter presented the complete technical design of SIPMS, covering the layered architecture, security model, all 16 entities with their relationships and attributes, four key sequence diagrams illustrating the main workflows, and the full database schema with 15 tables. These design decisions provide a solid blueprint for the implementation phase described in the next chapter.

Chapitre

4 IMPLEMENTATION

Sommaire

4.1 INTRODUCTION

This chapter presents the practical realization of the SIPMS platform. We describe the development environment and tools used, then walk through the four Scrum sprints, presenting the key technical components, backend services, frontend components, and delivered user interfaces for each sprint.

4.2 DEVELOPMENT ENVIRONMENT

4.2.1 Hardware Environment

Component	Specification
Processor	Intel Core i5 / i7, 2.4 GHz or higher
RAM	16 GB
Storage	512 GB SSD
Operating System	Windows 10 / 11 (64-bit)

TABLE 4.1 – Hardware development environment

4.2.2 Software Environment

Tool / Technology	Version	Purpose
Java (JDK)	17 LTS	Backend runtime
Spring Boot	3.2.x	Backend framework
Maven	3.9.x	Build and dependency management
MySQL	8.0	Relational database
Node.js	20 LTS	Frontend runtime
React (Vite)	18 / 5.x	Frontend framework
VS Code	Latest	Frontend IDE
IntelliJ IDEA	2023.x	Backend IDE
Postman	Latest	API testing
Git / GitHub	-	Version control
MySQL Workbench	8.0	Database management

TABLE 4.2 – Software development environment

4.3 SPRINT 1 — AUTHENTICATION AND USER MANAGEMENT

4.3.1 Backend Components Built

- **JwtTokenProvider** : A Spring `@Component` responsible for generating and validating HS512-signed JWT tokens. It reads the secret key and expiration duration from `application.properties`, builds the token with the user's email, ID, name, and roles in the claims, and provides a `validateToken()` method used by the security filter.
- **JwtAuthenticationFilter** : A `OncePerRequestFilter` that intercepts every HTTP request, extracts the `Authorization: Bearer` header, validates the token via `JwtTokenProvider` and populates the `SecurityContextHolder` with the authenticated user's details and granted authorities.
- **SecurityConfig** : A Spring Security configuration class that disables CSRF (stateless API), configures CORS to allow the frontend origin (`http://localhost:5173`), sets session management to `STATELESS`, defines public endpoints (`/api/auth/**`), and applies the `JwtAuthenticationFilter` before `UsernamePasswordAuthenticationFilter`
- **AuthController / AuthService** : Exposes `POST /api/auth/login` which accepts email and password, validates credentials via `AuthenticationManager`, and returns a signed JWT with user profile data and roles.
- **UserController / UserService** : Full CRUD at `/api/users` with `@PreAuthorize("hasRole('AD` enforcement. Uses BCrypt for password hashing (strength 10) and generates temporary passwords via `SecureRandom`.

4.3.2 Frontend Components Built

- **LoginPage.jsx** : A styled login form with email/password fields, error display, and loading state. On success, stores the JWT token in `localStorage` and dispatches a `LOGIN` action to `AuthContext`.
- **AuthContext.jsx** : A React context provider that manages authentication state (`isAuthenticated`, `user`, `token`), provides `login()` and `logout()` functions, and persists/restores state from `localStorage`.
- **RoleBasedRedirect** : A React component in `AuthContext` that reads the user's role from the JWT payload and redirects to the appropriate dashboard path (`/reception-panel` for RECEPTIONIST, `/quiz-interface` for CANDIDATE, `/dashboard` for ADMIN/MANAGER

- **ProtectedRoute.jsx** : A wrapper component that checks `isAuthenticated` and role membership before rendering child components. Unauthenticated users are redirected to `/login`.
- **UsersPage.jsx** : An admin-only page with a data table listing all users, a modal for creating/editing users with auto-generated passwords, and toggle-active functionality.

4.3.3 Delivered Interfaces

FIGURE 4.1 – Login page — SIPMS CLINISYS branding

FIGURE 4.2 – Administrator dashboard — Users management panel

4.4 SPRINT 2 — RECEPTION MODULE

4.4.1 Backend Components Built

- **Candidate entity** (standalone) : Refactored from the original design to be independent of the `User` entity. Contains `firstName`, `lastName`, `email`, `phone`, `cin` as direct fields with an optional `@OneToOne` link to `User`. Includes convenience getters (`getSkillsTags()`, `getDegree()`, `getUniversity()`, `getGraduationYear()`) that delegate to the latest `InternshipFile`.
- **InternshipFile entity** : A per-year dossier entity linked to `Candidate` via `@ManyToOne`. Stores `year`, `university`, `degree`, `skillsTags` and has a `@OneToMany` collection of `Document` records.
- **Document entity** : Stores file metadata (`fileName`, `filePath`, `fileType`, `fileSize`, `documentType`) with nullable `@ManyToOne` links to both `Application` and `InternshipFile`. The `application_id` column is nullable to allow documents linked exclusively to internship files.
- **FileStorageService** : A Spring `@Service` that stores uploaded files to `./uploads/{subdirectory` with UUID-based filenames. Returns relative paths for database persistence.
- **CandidateController endpoints** : `GET /api/candidates`, `POST /api/candidates` (creates standalone candidate), `POST /candidates/{id}/internship-files/with-document` (multipart upload creating both InternshipFile and Document records).

4.4.2 Frontend Components Built

- **ReceptionPanel.jsx** : A multi-tab dashboard with a candidates list (showing name, email, phone, CIN, user account status, file count), an internship files tab (listing all files across candidates), and a create-candidate modal. Includes search filtering by name/email and per-year filtering.
- **Add Internship File modal** : A form inside the reception panel with fields for year (dropdown), university, degree, skills tags (comma-separated), and a file upload input accepting PDF/DOC/DOCX. Uses `FormData` to send multipart requests to the backend.
- **CandidatesApi integration** : Axios API layer with `addInternshipFileWithDocument()` sending `multipart/form-data` and `addInternshipFile()` sending JSON, both returning the created `InternshipFileDto`.

4.4.3 Delivered Interfaces

FIGURE 4.3 – Reception Panel — Candidate list with year filter

FIGURE 4.4 – New Candidate registration modal with document upload

FIGURE 4.5 – Internship Files tab — All files across candidates

4.5 SPRINT 3 — QUIZ MODULE

4.5.1 Backend Components Built

- **Quiz entity** : Stores `title`, `description`, `durationMins`, `passingScore`, `totalMarks`, `specialty`, and a `@OneToMany` collection of `QuizQuestion`. Each quiz is optionally linked to a creator via `@ManyToOne User createdBy`.
- **QuizQuestion entity** : Stores `questionText`, four options (`optionA`–`optionD`), `correctOption`, `marks`, and `orderIndex`. Linked to `Quiz` via `@ManyToOne`.
- **QuizAttempt entity** : Records a candidate's attempt with `score`, `totalMarks`, `percentage`, `passed` boolean, `startedAt`, and `completedAt`. Linked to `Quiz`, `Candidate`, and `Application`.
- **CandidateService.approveAndSendQuiz()** : A transactional method that (1) creates a `User` account with BCrypt-hashed temp password, (2) assigns an existing quiz by `quizId` or creates a default 5-question MCQ quiz, (3) creates a `QuizAttempt`, (4) sends an HTML email via Gmail SMTP with login credentials and quiz title.
- **QuizService.submitQuiz()** : Receives submitted answers, compares each against the stored correct option, calculates the percentage score, updates the `QuizAttempt`, advances the `Application` status to `QUIZ_COMPLETED`, and returns the result.

4.5.2 Frontend Components Built

- **QuizSelectionModal.jsx** : A modal that loads existing quizzes from `GET /api/quizzes` and lets the manager select one or choose "Create Default" to generate a 5-question MCQ quiz. Used from both `CandidatesPage` and `OverviewPage`.
- **QuizInterface.jsx** : A quiz-taking component with a real-time countdown timer using `useEffect` and `setInterval`. Renders MCQ question cards with radio buttons. Auto-submits when timer reaches zero. Displays the score immediately after submission.
- **CandidatesPage.jsx** : A manager-facing page with a data table of candidates, "Approve & Send Quiz" button per candidate opening the `QuizSelectionModal`, and status indicators (user account active, quiz sent, etc.).

4.5.3 Delivered Interfaces

FIGURE 4.6 – Quiz interface — Question view with countdown timer

FIGURE 4.7 – Quiz selection modal — Manager chooses existing quiz or creates default

4.6 SPRINT 4 — INTERVIEW AND AI MODULE

4.6.1 Backend Components Built

- **Interview entity** : Stores `scheduledAt`, `interviewer`, `type` (TECHNICAL/HR), `status` (SCHEDULED/COMPLETED/CANCELLED), `score`, `notes`, and `feedback`. Linked via `@ManyToOne` to both `Candidate` and `Application`.
- **InterviewController** : Exposes `POST /api/interviews` (create/schedule), `GET /api/interviews` (list all), `GET /api/interviews/{id}`, `PUT /api/interviews/{id}` (update), `DELETE /api/interviews/{id}`, and `PUT /api/interviews/{id}/result` (record score, notes, feedback, update status to COMPLETED).

- **InterviewService** : Contains `createInterview()` with validation (scheduledAt must be in the future), `updateResult()` (validates status transition, updates score/notes/feedback), and `getInterviews()` (returns all interviews with candidate names).
- **AiRanking entity** : Stores AI-generated scores with `rankingType` (PROJECT_RANK / CANDIDATE_MATCH), `referenceId`, `score` (0.0–1.0), `reasoning` (AI explanation text), and an optional `@ManyToOne` to `Supervisor`.
- **AiService** : Implements `rankProjects()` which scores projects by relevance, and `matchCandidates()` which scores candidates for a given supervisor. Uses a weighted formula combining skill overlap, quiz performance, and academic level. Delegates to GPT-4-mini for reasoning generation.

4.6.2 Frontend Components Built

- **InterviewPage.jsx** : A manager-facing page listing all interviews with status badges. Includes a "Schedule Interview" modal (date/time picker, interviewer name, type dropdown) and a "Record Result" modal (score input, notes textarea, feedback textarea, status selector).
- **ApplicationsPage.jsx** : Extended with "Schedule Interview" button for applications in `QUIZ_COMPLETED`, `AI_EVALUATING`, or `MANAGER_REVIEW` status. Triggers the same scheduling flow.
- **Sidebar.jsx** : Updated with "Candidates" and "Quizzes" navigation links for the MANAGER role, and an "Interviews" link for interview management.
- **InterviewsApi** : Axios integration with six endpoints (`getAll`, `getById`, `create`, `update`, `delete`, `updateResult`) mapped to the `/api/interviews` resource.

4.6.3 Delivered Interfaces

FIGURE 4.8 – Interview management page — Schedule and record results

FIGURE 4.9 – AI Insights page — Project rankings and candidate matching

4.7 EVALUATION

This section evaluates the implementation against key quality metrics to assess how well the built system performs.

4.7.1 Code Quality

The backend codebase follows a strict layered architecture (Controller → Service → Repository) with clear separation of concerns. Key quality indicators :

- **Total backend classes** : 16 entities, 8 controllers, 8 services, 15+ DTOs, 12 repositories, 6 security/config classes.
- **Total frontend components** : 20+ page components, 15+ reusable UI components, 5 context providers, 3 API modules.
- **RESTful design** : All endpoints follow REST conventions with proper HTTP methods (GET, POST, PUT, PATCH, DELETE) and consistent response wrappers (`ApiResponse<T>`).
- **Error handling** : Global exception handler (`@ControllerAdvice`) catches all exceptions and returns structured JSON error responses with appropriate HTTP status codes.
- **Validation** : DTO-level validation using Jakarta `@NotBlank`, `@Email`, `@Size` annotations with automatic 400 Bad Request responses.

4.7.2 API Performance

The key API endpoints were benchmarked using Postman with 10 sequential requests per endpoint :

Endpoint	Avg (ms)	Max (ms)	Status
POST /api/auth/login	245	412	Pass (< 2s)
GET /api/candidates	180	310	Pass (< 2s)
POST /api/candidates	320	580	Pass (< 2s)
POST /.../internship-files/with-document	850	1450	Pass (< 2s)
POST /api/quiz/submit	420	780	Pass (< 2s)
GET /api/interviews	195	340	Pass (< 2s)

TABLE 4.3 – API endpoint performance benchmarks

4.7.3 Security Assessment

- **Authentication** : JWT tokens signed with HS512 (256-bit secret), 24-hour access token expiry, 7-day refresh token expiry.
- **Password storage** : All passwords hashed with BCrypt (strength 10) — no plain-text storage anywhere.
- **Authorization** : Every endpoint guarded by `@PreAuthorize` — tested with a matrix of 4 roles $\times$ 10 endpoint groups, all returning correct HTTP 200 or 403.
- **File upload** : Files stored outside the application root in `./uploads/` with UUID-based names to prevent path traversal. Type restricted to PDF/DOC/DOCX, size limited to 10 MB.

4.7.4 Coverage Summary

The implementation delivers all 21 user stories defined in the product backlog (US-01 through US-21), covering authentication, candidate management, internship files, quiz assignment and grading, interview scheduling, AI-powered matching, and automated email notifications.

4.8 CONCLUSION

This chapter presented the complete implementation of the SIPMS platform across four Scrum sprints. Each sprint delivered working, tested software increments. The evaluation confirms that the system meets all functional requirements, performs within acceptable response time thresholds, and enforces security at every layer. The combination of a Spring Boot backend, React frontend, MySQL database, and AI integration produces a robust platform ready for production deployment at CLINISYS.

Chapitre

5 TESTING AND VALIDATION

Sommaire

5.1 INTRODUCTION

The purpose of testing is to ensure that the SIPMS platform meets the specified functional and non-functional requirements, operates reliably under expected loads, and maintains security against unauthorized access. This chapter presents the comprehensive testing strategy applied across all modules, the test cases executed, performance benchmarks, security validation, and a summary of results demonstrating that all requirements are satisfied.

5.2 TESTING STRATEGY

SIPMS was validated using a four-tier testing pyramid to ensure coverage at every level of the system :

5.2.1 Unit Testing

Individual components — services, controllers, utilities, and helpers — were tested in isolation using JUnit 5 and Mockito. Each method was verified for correct output given known inputs, edge cases, and error conditions. Key modules tested :

- **Authentication Service** : Login validation, token generation and parsing, role extraction, BCrypt password hashing, refresh token logic.
- **Candidate Service** : CRUD operations, file upload and path generation, quiz assignment, email trigger conditions.
- **Quiz Service** : Question scoring algorithms, attempt creation and expiration, percentage calculation, pass/fail determination.
- **Interview Service** : Scheduling validation, status transitions (SCHEDULED $\rightarrow$ COMPLETED $\rightarrow$ CANCELLED), score recording.
- **Notification Service** : Creation, retrieval, unread count, mark-as-read logic.

5.2.2 Integration Testing

All REST API endpoints were tested end-to-end using Postman collections that simulate real client requests. Each endpoint was validated against the following criteria :

- Correct HTTP status codes for success (200, 201), client errors (400, 401, 403, 404), and server errors (500).
- Request payload validation — missing required fields, invalid data types, boundary values.
- Authentication and authorization — unauthenticated requests rejected, role-based access enforced (RECEPTIONIST cannot access MANAGER endpoints).
- File upload and download flows — multipart requests, content-type validation, file size limits.
- Database transaction integrity — rollback on failure, constraint enforcement.

5.2.3 System Testing

The complete platform — frontend + backend + database — was tested as a unified system. End-to-end user journeys were executed in a staging environment mirroring production :

- Full candidate lifecycle : Registration → File Upload → Quiz Assignment → Quiz Completion → Manager Review → Acceptance.
- Cross-role workflows : Receptionist creates candidate, Manager approves and sends quiz, Candidate takes quiz, Manager interviews.
- Concurrent session handling : multiple users interacting simultaneously without data corruption.

5.2.4 User Acceptance Testing (UAT)

The platform was demonstrated to CLINISYS stakeholders (managers and receptionists) who validated each workflow against real operational scenarios. Feedback was collected and incorporated into the final sprint. All critical user stories (US-01 through US-21) were accepted.

5.3 TEST CASES

Test ID	Feature	Test Scenario	Expected Result
TC-01	Authentication	Login with valid credentials	JWT token returned, user redirected to role-based dashboard
TC-02	Authentication	Login with invalid password	401 Unauthorized, error message displayed
TC-03	Authentication	Access protected endpoint without token	403 Forbidden
TC-04	Role-Based Access	RECEPTIONIST accesses /api/candidates	200 OK, candidate list returned
TC-05	Role-Based Access	RECEPTIONIST accesses /api/admin/users	403 Forbidden
TC-06	Candidate Creation	Create candidate with valid data (firstName, lastName, email, phone, CIN)	200 OK, candidate created with ID
TC-07	Candidate Creation	Create candidate with missing email	400 Bad Request, validation error
TC-08	Internship File	Add internship file with document upload	200 OK, file stored, Document record created
TC-09	Internship File	Add internship file without document	200 OK, file record created with empty documents
TC-10	Quiz Assignment	Approve candidate and send default quiz	200 OK, User account created, email sent with credentials
TC-11	Quiz Assignment	Approve candidate with existing quizId	200 OK, existing quiz assigned to candidate
TC-12	Quiz Taking	Submit quiz with all correct answers	Score calculated, status set to QUIZ_COMPLETED
TC-13	Quiz Taking	Submit quiz after 48-hour expiry	Attempt rejected, score set to 0
TC-14	Interview	Schedule interview with valid data	200 OK, Interview created with SCHEDULED status
TC-15	Interview	Record interview result with score	200 OK, status updated to COMPLETED, score saved
TC-16	File Download	Access uploaded PDF via /uploads/... URL	200 OK, PDF served with correct Content-Type
TC-17	Email Notification	Manager approves candidate	Email sent to candidate with login credentials
TC-18	User Management	Admin creates new user account	200 OK, user created, password generated

TABLE 5.1 – Test case summary — key scenarios

5.4 PERFORMANCE TESTING

Performance testing was conducted to verify that the system meets responsiveness requirements under normal and peak loads.

5.4.1 Response Time Validation

Key API endpoints were benchmarked using Postman with 10 sequential requests. The average response time was measured and validated against the 2-second threshold :

Endpoint	Avg Response (ms)	Max (ms)	Status
POST /api/auth/login	245	412	Pass (< 2s)
GET /api/candidates	180	310	Pass (< 2s)
POST /api/candidates	320	580	Pass (< 2s)
POST /api/candidates/{id}/internship-files/with-document	850	1450	Pass (< 2s)
POST /api/quiz/submit	420	780	Pass (< 2s)
GET /api/interviews	195	340	Pass (< 2s)

TABLE 5.2 – API response time benchmarks

All endpoints responded well within the 2-second threshold, confirming that the system is performant for real-time use.

5.4.2 Load Handling

The system was tested with up to 20 concurrent simulated users performing simultaneous operations. Connection pool settings (HikariCP max 10) and thread pool configurations handled the load without timeout or data corruption.

5.5 SECURITY TESTING

Security was validated across multiple dimensions to ensure data protection and access control.

5.5.1 Role-Based Access Control (RBAC)

Each endpoint in the system is annotated with `@PreAuthorize` to enforce role restrictions. The following matrix was tested :

Endpoint Group	Admin	Manager	Receptionist	Candidate
GET /api/users		403	403	403
GET /api/candidates				403
POST /api/candidates/id/approve-and-send-quiz			403	403
POST /api/interviews			403	403
POST /api/quiz/submit	403	403	403	

TABLE 5.3 – RBAC access matrix — tested and verified

5.5.2 JWT Validation

- **Token generation** : Tokens are signed using HS512 algorithm with a 256-bit secret key and include expiration timestamp.
- **Token expiration** : Access tokens expire after 24 hours ; refresh tokens after 7 days. Expired tokens are rejected with 401.
- **Token tampering** : Modified tokens are detected during signature verification and rejected.
- **Password security** : All passwords are hashed using BCrypt with strength factor 10.

5.6 VALIDATION RESULTS

The following table maps each functional requirement to its validation status, demonstrating that all requirements are satisfied :

US ID	Requirement	Test Coverage	Status
US-01	Role-based login	TC-01 — TC-05	Verified
US-02	Admin creates users	TC-18	Verified
US-03	Password change on first login	Integration tested	Verified
US-04	Receptionist registers dossiers	TC-06 — TC-09	Verified
US-05	Admin/Manager manages users	TC-18	Verified
US-06	Supervisor management	Integration tested	Verified
US-07	Candidate submits application	Integration tested	Verified
US-08	AI ranks projects	Integration tested	Verified
US-09	AI suggests supervisor	Integration tested	Verified
US-10	Timed technical quiz	TC-12, TC-13	Verified
US-11	Auto-correct quiz	TC-12	Verified
US-12	Specialty-based quiz (48h)	TC-13	Verified
US-13	Manager sets candidate status	Integration tested	Verified
US-14	Automatic email notifications	TC-17	Verified
US-15	Role-based dashboard redirect	TC-01, TC-04, TC-05	Verified
US-16	Per-year internship files	TC-08, TC-09	Verified
US-17	View all internship files	TC-08	Verified
US-18	Approve and send quiz	TC-10, TC-11	Verified
US-19	Schedule interview	TC-14	Verified
US-20	Record interview result	TC-15	Verified
US-21	View all interviews	TC-14	Verified

TABLE 5.4 – Requirements traceability matrix — all user stories verified

5.7 CONCLUSION

The testing phase confirmed that all 21 user stories are fully implemented and validated. The system handles edge cases gracefully, enforces role-based access at every endpoint, and responds within acceptable performance thresholds. No critical or high-severity defects remain.

Chapitre

6 DISCUSSION AND EVALUATION

Sommaire

6.1 INTRODUCTION

This chapter evaluates the SIPMS platform against the initial objectives defined at the project outset. We discuss the strengths and limitations of the system, analyze the chosen technologies, and propose future enhancements.

6.2 EVALUATION OF OBJECTIVES

Objective	Status	Achievement
Multi-role authentication with RBAC	Achieved	JWT + Spring Security
Candidate registration and file management	Achieved	Reception Panel
Automated quiz assignment and grading	Achieved	Quiz System
AI-powered project ranking	Achieved	Spring AI Integration
Interview scheduling and result tracking	Achieved	Interview Module
Email notifications	Achieved	JavaMail + Gmail SMTP
Supervisor assignment and capacity management	Achieved	Supervisor Module

TABLE 6.1 – Evaluation of project objectives

6.3 TECHNOLOGY EVALUATION

6.3.1 Backend : Spring Boot 3.2 + Java 17

Spring Boot provided a robust foundation with built-in security, JPA integration, and REST API support. The layered architecture (Controller $\rightarrow$ Service $\rightarrow$ Repository) ensured clear separation of concerns and testability.

6.3.2 Frontend : React 19 + Vite

React's component-based model enabled rapid UI development. Vite's fast build times and HMR significantly improved development productivity. Tailwind CSS allowed consistent styling without writing custom CSS.

6.3.3 Database : MySQL 8

MySQL proved reliable for storing structured relational data. Hibernate's DDL auto-update simplified schema evolution during development.

6.3.4 AI Integration

Spring AI provided seamless integration with GPT-4-mini for project ranking and supervisor matching. The AI module operates as a recommendation engine rather than a decision maker, preserving human oversight.

6.4 LIMITATIONS

- **AI dependency on external API** : The AI module requires a stable internet connection and an active API key. Offline fallback is not implemented.
- **Email delivery reliability** : Gmail SMTP has daily send limits that could be reached in high-volume deployments.
- **File storage** : Currently uses local filesystem storage. A cloud storage solution (AWS S3, MinIO) would be more scalable.
- **Real-time notifications** : The system uses polling instead of WebSockets for in-app notifications, which may introduce slight delays.

6.5 FUTURE ENHANCEMENTS

- **WebSocket integration** for real-time notification delivery

- **Cloud file storage** with AWS S3 or MinIO for better scalability
- **Multi-language support** for international internship programs
- **Advanced analytics dashboard** with charts and exportable reports
- **Mobile application** using React Native for on-the-go access
- **Calendar integration** with Google Calendar / Outlook for interview scheduling

6.6 CONCLUSION

The SIPMS platform successfully meets all core requirements defined at the start of the project. While some limitations exist, the system is production-ready and deployed at CLINISYS. The modular architecture allows easy addition of future enhancements.

GENERAL CONCLUSION

This final-year project resulted in the successful design and development of **SIPMS (Smart Internship and Project Management System)**, a comprehensive full-stack web platform built for **CLINISYS** to digitize and automate the complete internship management lifecycle.

Throughout this project, we addressed a real organizational need : replacing a fragmented, manual internship management workflow with a structured, secure, and intelligent digital platform. The system was built following the **Scrum agile methodology**, delivering four functional sprints that progressively implemented all required features.

The key achievements of this project are :

- **Secure multi-role authentication** : A stateless JWT-based system with BCrypt password hashing and fine-grained RBAC enforced across all API endpoints.
- **Automated candidate onboarding** : The Reception Panel allows receptionists to register candidates, upload multiple documents, filter by internship year, and trigger automated welcome emails — all within a single workflow.
- **Objective competency evaluation** : A fully configurable quiz system with a timed interface, automatic scoring, and status management ensures fair and consistent candidate assessment.
- **AI-powered decision support** : Project ranking and candidate-to-supervisor matching algorithms provide managers with data-driven recommendations, reducing subjectivity in assignment decisions.
- **Complete application lifecycle** : From physical reception to final supervised project assignment, every stage of the internship process is tracked, versioned, and auditable.

From a technical perspective, this project provided valuable hands-on experience with a modern, production-grade technology stack : **React (Vite)** for a reactive and accessible frontend, **Spring Boot (Java 17)** for a robust and scalable backend, **MySQL** for relational data management,

and **Spring Security + JWT** for enterprise-grade authentication. The integration of **JavaMail** for automated notifications and **Spring Multipart** for document management further enriched the technical scope of the project.

Looking ahead, several enhancements could further strengthen the SIPMS platform :

- **Mobile application** : A React Native or Flutter companion app allowing candidates to track their status and receive push notifications on the go.
- **Advanced AI models** : Replacing the current scoring engine with machine learning models trained on historical internship outcome data to produce more accurate matchings.
- **Electronic signature** : Integrating a digital contract signing module to fully dematerialize the internship agreement workflow.
- **Analytics dashboard** : A management reporting module with visual KPIs tracking acceptance rates, quiz performance distributions, and supervisor workload trends.
- **Docker containerization** : Packaging the entire stack (frontend, backend, database) into Docker Compose for simplified deployment and horizontal scalability.

In conclusion, the SIPMS platform successfully delivers on its core objective : transforming a manual, error-prone process into an efficient, transparent, and intelligent system that benefits all stakeholders — from the receptionist registering a candidate on day one, to the manager making a data-informed project assignment decision weeks later. This project stands as a solid foundation for continued digital innovation within CLINISYS.

Bibliographie

[1] Pivotal Software, *Spring Boot Reference Documentation*, version 3.2, Available at : https://docs.spring.io/spring-boot/docs/current/reference/html/, 2024.

[2] Pivotal Software, *Spring Security Reference*, version 6.x, Available at : https://docs.spring.io/spring-security/reference/, 2024.

[3] Meta Platforms Inc., *React Documentation*, Available at : https://react.dev/, 2024.

[4] M. Jones, J. Bradley, N. Sakimura, *JSON Web Token (JWT)*, RFC 7519, Internet Engineering Task Force (IETF), May 2015.

[5] Evan You, *Vite — Next Generation Frontend Tooling*, Available at : https://vitejs.dev/, 2024.

[6] Oracle Corporation, *MySQL 8.0 Reference Manual*, Available at : https://dev.mysql.com/doc/refman/8.0/en/, 2024.

[7] Red Hat Inc., *Hibernate ORM Documentation*, version 6.x, Available at : https://hibernate.org/orm/documentation/, 2024.

[8] Oracle Corporation, *JavaMail API Documentation*, Available at : https://javaee.github.io/javamail/, 2023.

[9] N. Provos, D. Mazières, *A Future-Adaptable Password Scheme*, Proceedings of USENIX Annual Technical Conference, 1999.

[10] K. Schwaber, J. Sutherland, *The Scrum Guide — The Definitive Guide to Scrum : The Rules of the Game*, November 2020, Available at : https://scrumguides.org/.

[11] Object Management Group (OMG), *Unified Modeling Language (UML) Specification*, version 2.5.1, Available at : https://www.omg.org/spec/UML/, 2017.

[12] R. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, PhD Dissertation, University of California, Irvine, 2000.

[13] Lucide Contributors, *Lucide React — Beautiful & consistent icon toolkit*, Available at : https://lucide.dev/, 2024.

ANNEXES

A.1 TITRE ANNEXE 1

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam. Maecenas ligula massa, varius a, semper congue, euismod non, mi. Proin porttitor, orci nec nonummy molestie, enim est eleifend mi, non fermentum diam nisl sit amet erat. Duis semper. Duis arcu massa, scelerisque vitae, consequat in, pretium a, enim.

FIGURE A.1 – Titre de figure (exemple)

Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique
Institut International de Technologie de
Sfax

وزارة التعليم العالي والبحث العلمي
المدرسة العليا الدولية الخاصة للتكنولوجيا بصفا قس

TITRE DU PROJET

Prénom NOM _____

Résumé :

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam. Maecenas ligula massa, varius a, semper congue, euismod non, mi. Proin porttitor, orci nec nonummy molestie, enim est eleifend mi, non fermentum diam nisl sit amet erat. Duis semper. Duis arcu massa, scelerisque vitae, consequat in, pretium a, enim. Pellentesque congue.

Mots clés : Lorem, ipsum, dolor, sit amet, consectetur, adipiscing elit.

Abstract :

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam. Maecenas ligula massa, varius a, semper congue, euismod non, mi. Proin porttitor, orci nec nonummy molestie, enim est eleifend mi, non fermentum diam nisl sit amet erat. Duis semper. Duis arcu massa, scelerisque vitae, consequat in, pretium a, enim. Pellentesque congue.

Key-words : Lorem, ipsum, dolor, sit amet, consectetur, adipiscing elit.